

FinQuery 架构学习笔记：从检索、MCP 到 LangGraph

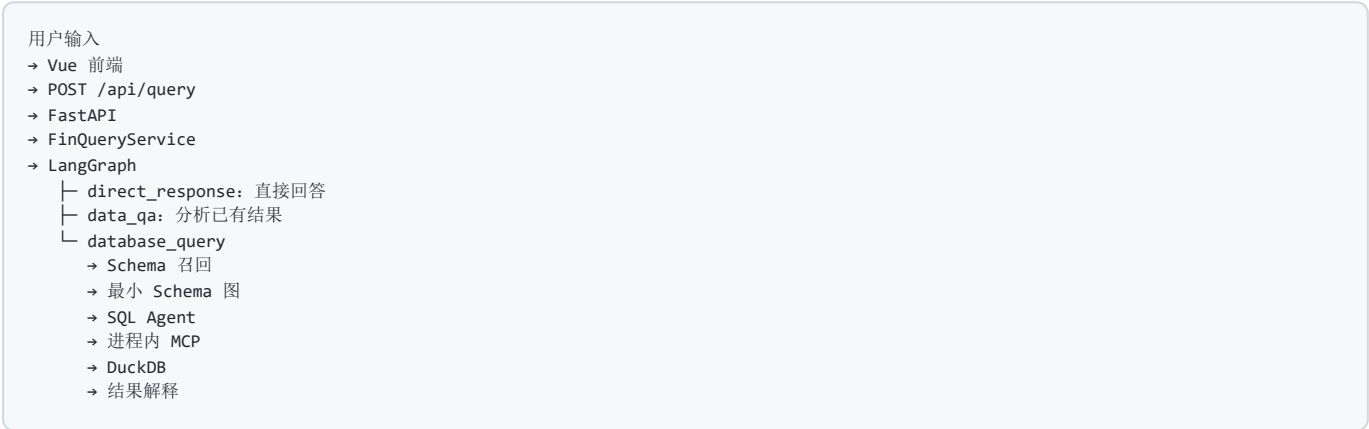
整理日期：2026-09-23

项目：D:\FinQuery

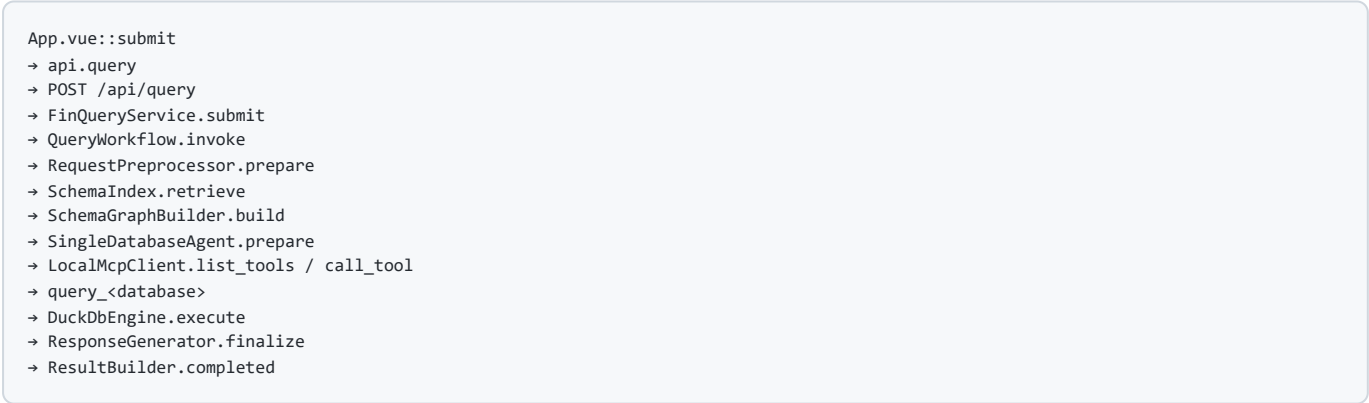
依据：本次对话的问题线索与当前项目源码。实现行为以源码为准，协议内容按当前公开规范校正。

1. 整体架构

FinQuery 是面向金融股票数据的自然语言查询系统。用户输入中文问题后，系统判断这是普通问答、已有数据分析，还是需要查询数据库；数据库问题先召回相关 Schema，再由 Agent 生成只读 SQL，通过 MCP 工具调用 DuckDB，最后将结果整理成回答。



普通数据库查询的核心调用链：



入口文件：

- 前端提交 App.vue
- HTTP 客户端 api.ts
- 服务入口 finquery_service.py
- LangGraph 主图 query_graph.py

2. 项目命名

原名 askData 对金融股票数据库的定位不够清楚，因此选择 FinQuery：

- **Fin**：Finance / Financial Data。
- **Query**：查询、分析、自然语言问数。
- 能覆盖股票行情、财务报表、指数、ETF 等数据。

本次对话的约定是采用 **FinQuery**，命名更新由用户手动完成，不自动改代码。

3. 什么是召回阶段

召回阶段的目标不是立即产生最终答案，而是：

从大量候选内容中尽可能找出相关内容，组成较小候选集，交给后续重排和推理。

用户问：

查询贵州茅台最近 5 个交易日的收盘价和成交量

Schema 召回希望找到：

```
stock_daily.ts_code
stock_daily.trade_date
stock_daily.close
stock_daily.vol
```

召回阶段强调“不要漏掉”，后续 Rerank 强调“把最相关的排前面”。FinQuery 的管线是：

```
问题 + retrieval_terms
├─ BM25 关键词召回
└─ Embedding 语义召回
    ↓
    RRF 融合
    ↓
    模型 Rerank
    ↓
    阈值过滤
    ↓
    最终 Schema 字段
```

重点代码：

- [SchemaIndex.retrieve](#)
- [BM25 与 Dense 检索](#)
- [检索词提取](#)

4. RRF 排序

RRF 是 Reciprocal Rank Fusion，**倒数排名融合**。它把多个检索器的排名合并，不要求各检索器的原始分数处于同一量纲。

$$\text{RRF}(d) = \sum 1 / (k + \text{rank}_i(d))$$

FinQuery 使用 `k = 60`：

```
fused[index] += 1 / (60 + rank)
```

代码：[retrieval/service.py](#)

例子：

字段	BM25 名次	Dense 名次	RRF 思路
close	1	3	两边都靠前
vol	3	1	两边都靠前
trade_date	2	2	两边排名稳定

RRF 之后仍然有模型 Rerank，所以 RRF 是候选融合，不是最终相关性判断。

5. HNSW、IVF 与索引建立时机

HNSW

HNSW（Hierarchical Navigable Small World）把向量组织成多层图：高层快速跳转，底层局部精细搜索。

- 查询快、召回率通常高。
- 内存占用较大。
- 构建索引相对昂贵。
- 适合频繁在线查询。

IVF

IVF（Inverted File Index）先聚类，把向量分配到不同桶。查询时只搜索最接近的若干桶。

- 适合大规模数据。
- 通常比 HNSW 节省内存。
- 质量取决于聚类和 `nprobe`。
- 大量数据变化后可能需要重新训练聚类中心。

什么时候建立

通常在数据准备或索引更新阶段建立，而不是每次查询时建立：

内容变化

→ 生成文本

→ Embedding

→ 建立/更新 HNSW 或 IVF

→ 查询阶段直接使用

HNSW 可随插入逐步扩展图；IVF 通常先训练聚类中心，再分配向量。

FinQuery 当前实现

FinQuery 当前没有 HNSW、IVF 或 Milvus。它把 Schema 向量保存在本地 JSON 索引中，查询时对有权限的字段执行全量点积：

```
sum(left * right for left, right in zip(query_vector, document.dense_vector))
```

代码：[retrieval/store.py](#)

`SchemaIndex.ensure_built()` 会检查 Schema 签名和 Embedding 模型；缓存失效时重新生成字段文档、向量并保存为：

```
backend/data/schema_store.<database-signature>.json
```

代码：[retrieval/service.py](#)

6. SYNONYMS 和模型分别参与什么

SYNONYMS 扩展字段的业务表达，例如“营业收入”“营收”。定义见 [database_sources/trade_data.py](#)。

构建检索文档时，同义词加入：

- `keyword_text`：BM25 使用。
- `semantic_text`：Embedding 使用。
- `rerank_text`：Rerank 使用。

代码：[retrieval/service.py](#)

以下上下文由 Python 按 Session/Workspace 确定性生成，不使用 **SYNONYMS**，构造当下也不会发起新的模型调用：

- `route_context`
- `recent_result_context`
- `analysis_context`
- `analysis_sources`

模型参与的位置：

- `RequestPreprocessor.prepare()`：路由、改写、检索词。
- `SchemaIndex`：Embedding、Rerank。
- `SingleDatabaseAgent.prepare()`：工具选择和 SQL。
- `DataQaAgent.run()`：分析已有结果。
- `ResponseGenerator.finalize()`：结果解释。
- `ShortTermMemory`：超过阈值后异步总结旧轮次。

7. MCP 的正确理解

7.1 MCP 不等于 Agent-to-Agent

MCP 是 **Model Context Protocol**，定义 Agent Host 如何发现和调用外部工具、资源、提示词等能力。

REST：普通程序调用服务的接口契约
MCP：Agent Host 获取外部能力的标准协议

MCP Server 可以包装数据库、文件、REST API、SaaS、网页后端、搜索服务，也可以包装另一个 Agent；它本身不一定包含模型。

7.2 Host、Client、Server



- Host：承载 Agent 的应用。
- Client：Host 内负责 MCP 通信的组件。
- Server：对外发布工具、资源或提示词。

7.3 远程 MCP 与 REST

对比	REST API	远程 MCP
面向	普通应用	Agent Host / 模型工具系统
描述	OpenAPI/文档	<code>tools/list</code> 等机器可读 Schema
调用	URL + HTTP 方法	<code>tools/call</code> 等 MCP RPC
选择接口	程序预先编码	模型可结合描述选择工具
上下文类型	自定义	Tools、Resources、Prompts
认证	API Key/OAuth 等	同样需要 OAuth/Bearer 等

常见组合是：

Agent → MCP Server → 现有 REST API → 数据库

7.4 远程 Server 如何发现

要区分两层：

1. **发现服务器地址**：手工配置、产品目录、企业私有 Registry、官方/第三方 Registry 或管理员下发。
2. **发现服务器能力**：已经知道端点后，再调用 `server/discover`（当前协议可选）、`tools/list`、`resources/list`、`prompts/list`。

`server/discover` 不是互联网搜索；它只询问一个已经知道地址的 Server。公开 Server 可通过 [Official MCP Registry](#) 查找。

7.5 当前协议与旧协议

截至 2026-09-23，MCP 2026-07-28 使用无状态核心：

- 不再要求旧版 `initialize` / `initialized`。
- 不再依赖 `Mcp-Session-Id`。
- 请求携带协议版本、Client 身份和能力元数据。
- `server/discover` 是可选的能力发现。
- Streamable HTTP 使用 `Mcp-Method`、`Mcp-Name` 等请求头辅助路由。

旧版 2025-11-25 及更早实现仍可能通过 `initialize` 建立会话，接入时要确认版本。参考：[MCP 2026-07-28 发布说明](#) 与 [规范变更](#)。

7.6 Host 中通常配置什么

不同产品的配置格式不统一，但通常包括：

```
{
  "id": "financial-data",
  "display_name": "Financial Data MCP",
  "endpoint": "https://example.com/mcp",
  "transport": "streamable_http",
  "auth": {"type": "oauth2"},
  "enabled": true,
  "permission_policy": {
    "allow_tools": ["query_stock"]
  }
}
```

包括 Server 标识、Endpoint、Transport、认证引用、TLS/代理、启用状态、工具权限、批准策略、超时和重试。具体工具名称及参数应通过协议发现，而不是只靠静态配置。

7.7 工具调用示例

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "query_stock",
    "arguments": {
      "symbol": "600519.SH",
      "start_date": "2026-09-01"
    }
  }
}
```

7.8 FinQuery 当前是进程内 MCP

```
SingleDatabaseAgent
→ LocalMcpClient
→ create_local_mcp_server
→ query_trade_data
→ DuckDbEngine.execute
```

重点代码：

- [MCP Client](#)
- [MCP Server](#)
- [数据库工具](#)
- [SQL Agent](#)

进程内 MCP 是集成架构，不是安全控制；安全来自用户权限、工具白名单、SQL 校验和数据库授权。

8. 前端发送给后端的数据

```
{
  "query": "查询贵州茅台最近5个交易日收盘价",
  "session_id": "前端会话UUID",
  "workspace": {
    "schema_fields": [],
    "analysis_table_ids": [],
    "analysis_tables": []
  }
}
```

- [前端类型](#)
- [后端请求模型](#)
- [请求发送](#)

`workspace` 不是完整数据库或整段历史，而是用户明确选中的工作上下文：

- `schema_fields`：确认参与新查询的字段。
- `analysis_table_ids`：选中的历史结果 ID。
- `analysis_tables`：对应列和有限行数据，前端当前最多发送 50 行。

后端的 `query_workspace()` 只保留数据库查询所需的字段、表和参数；分析表拆成 `analysis_context` 与 `analysis_sources`。

- [前端组装 Workspace](#)
- [后端过滤 Workspace](#)
- [Payload 组装](#)

9. Session ID 生命周期

前端新建对话时用 `crypto.randomUUID()` 生成 ID，并在该对话所有请求中复用。若已有未使用的空对话，则直接选择它。 [代码](#)

后端隔离用户：

```
scoped_session_id = f"{access_scope.user_id}:{session_id}"
```

[代码](#)

当前没有后端 Session TTL 或显式回收接口：

- 删除前端对话只影响 UI。
- 前端最多保留 30 个对话，不会清理后端状态。
- 登出只清认证 Token。
- 后端重启会清空进程内状态。
- 启用 Archive 后可恢复已完成轮次和摘要。
- 正在等待澄清的 LangGraph checkpoint 不能靠 Archive 恢复。

10. 短期记忆与长期记忆

10.1 短期记忆在哪里

Agent 短期记忆主要在后端进程：

```
SessionContext.tasks  
SessionContext.session_tasks  
ShortTermMemory._states
```

- [SessionContext](#)
- [ShortTermMemory](#)

默认上下文约超过 12,000 tokens 后，系统异步总结旧轮次，并至少保留最近 5 轮。 [配置](#)

10.2 `route_context` 存在哪里

它不是独立存储的一段文本，而是每次请求从：

动态生成。默认最近 6 轮，并补充 Workspace 选择的表格标题。[代码](#)

10.3 各 Context

字段	内容	构造时调用模型？
<code>route_context</code>	最近几轮任务的紧凑路由信息	否
<code>short_term_context</code>	历史摘要 + 未摘要轮次	读取时否；生成旧摘要时会
<code>recent_result_context</code>	最近一次成功数据库结果	否
<code>analysis_context</code>	用户选中的历史表格和数据	否
<code>analysis_sources</code>	选中表格的来源元数据	否

代码：

- [route_context](#)
- [short_term_context](#)
- [recent_result_context](#)
- [analysis_context](#)

这些内容可能包含以前由模型生成的分析文本，但不代表本次构造 Context 时调用模型。

10.4 可选 Archive

默认 `SESSION_ARCHIVE_ENABLED=false`。启用后，已完成轮次、消息和摘要写入 `backend/data/session_archive.db`。[代码](#)

10.5 长期记忆何时进入

长期记忆来自用户主动保存，位于 `backend/data/saved_memories.json`。[代码](#)

- 已保存字段只有被选入 Workspace 才进入 `schema_fields`。
- 已保存结果只有被选入分析区才进入 `analysis_context`。
- 当前没有长期记忆的自动向量召回，也不会把所有保存内容自动注入 Prompt。

11. LangGraph 最初拿到的 State

`FinQueryService._payload()` 构造：

```
{
  "task_id": "a1b2c3d4e5f6",
  "query": "查询贵州茅台最近5个交易日收盘价",
  "session_id": "user-id:frontend-session-uuid",
  "workspace": {
    "schema_fields": [],
    "confirmed_schema_tables": [],
    "confirmed_parameters": {}
  },
  "access_scope": {
    "user_id": "...",
    "roles": [],
    "allowed_databases": [],
    "allowed_tables": []
  },
  "route_context": "JSON字符串",
  "short_term_context": "JSON字符串",
  "recent_result_context": "JSON字符串",
  "analysis_context": "JSON字符串",
  "analysis_sources": [],
  "tool_facts": {},
  "execution_log": [],
  "tool_calls": [],
  "clarification": null
}
```

运行配置：

```
{"configurable": {"thread_id": task_id}}
```

`session_id` 用于多轮业务上下文；`task_id` 代表一次请求，并作为 LangGraph checkpoint 的 `thread_id`。

完整类型：[workflows/state.py](#)

后续节点逐步加入：

```
intent, standalone_query, extraction, retrieval,
schema_graph, schema_context, database_names,
mcp_execution, direct_sql, sql_source, result
```

12. LangGraph 的本质

LangGraph 是：

以 State 为中心、用有向图组织节点、支持条件分支与循环的 Agent workflow 状态机。

它不一定是 DAG。FinQuery 有明确循环：

```
retrieve_schema
→ prepare_single_database
→ 信息不足
→ human_clarification
→ 用户回答
→ retrieve_schema
```

可以理解成：

参考：[LangGraph 官方参考](#)。

13. 节点函数与 Hook

业务函数注册为节点：

```
builder.add_node("preprocess", self._preprocess)
builder.add_node("retrieve_schema", self._retrieve_schema)
```

业务只调用一次：

```
self.graph.invoke(payload, config=...)
```

随后 LangGraph 自动调用节点、合并 State、按边选择下一节点。因此节点更像业务处理器，不是传统 Hook。

真正的 Callback/Hook 用于日志、Tracing、工具事件以及 `interrupt` / `resume` 生命周期观察。参考：[Graph callbacks](#)。

图定义：[query_graph.py](#)

14. LangGraph 后续调用了什么

14.1 `_preprocess`

```
_preprocess → RequestPreprocessor.prepare
```

模型完成上下文聚合、追问改写、意图路由和检索词提取，产生：

```
direct_response / data_qa / database_query
```

- [节点](#)
- [预处理器](#)

14.2 `direct_response`

```
_respond_directly → ResultBuilder.direct_response → END
```

预处理阶段已经生成回答，这里主要包装结果。

14.3 data_qa

```
_answer_qa → DataQaAgent.run → 可选 MCP 展示工具 → ResultBuilder.qa → END
```

它分析已有结果，不重新查股票数据库。[代码](#)

14.4 database_query

```
_retrieve_schema  
→ SchemaIndex.retrieve  
→ SchemaIndex.include_workspace  
→ SchemaGraphBuilder.build  
→ SchemaGraphBuilder.context_text
```

单库路径：

```
_prepare_single_database  
→ SingleDatabaseAgent.prepare  
→ LocalMcpClient.list_tools  
→ 模型生成工具调用和 SQL  
→ LocalMcpClient.call_tool  
→ query_<database>  
→ DuckDbEngine.execute
```

需要澄清时：

```
_human_clarification  
→ interrupt  
→ 用户回答  
→ Command(resume={"option_id": ...})  
→ 更新 workspace  
→ 重新 retrieve_schema
```

完成后：

```
_execute_single_database  
→ 整理 MCP trace 和 SqlExecution  
→ ResponseGenerator.finalize  
→ ResultBuilder.completed  
→ END
```

重要细节：实际 SQL 已在 `SingleDatabaseAgent.prepare()` 内通过 MCP 执行；
`_execute_single_database()` 主要整理返回值和生成最终说明。多数据库路径当前返回“尚未启用”。

15. 最小 Schema 图

最小 Schema 图是：

从完整 Schema 中只保留回答当前问题必需的表、字段和连接关系，形成足以生成正确 SQL 的最小结构上下文。

它不是完整 ER 图、实际数据或 SQL 执行计划。

单表示例：

用户：查询贵州茅台最近 5 个交易日收盘价和成交量

stock_daily

└ ts_code

└ trade_date

└ close

└ vol

过滤股票

日期过滤和排序

返回指标

返回指标

多表示例：

stock_basic

└ name

└ industry

└ ts_code

stock_daily

└ ts_code

└ trade_date

└ close

JOIN

即使用户不要求展示 `ts_code` ，它也是 JOIN 所必需的字段。

展示字段

+ 过滤字段

+ 聚合/分组字段

+ 排序字段

+ JOIN 字段

= 最小但足够的 Schema 图

- [SchemaGraphBuilder.build](#)
- [context_text](#)

16. SQL 安全边界

`DuckDbEngine` 使用 `SQLGlot` 检查：

- 只允许一条查询语句。
- 拒绝写入和危险操作。
- 拒绝 `SELECT *` 。
- 检查已注册表。
- 检查用户数据库/表权限。
- 最多返回 200 行。
- [执行入口](#)
- [SQL 校验](#)

MCP 负责能力暴露和调用标准化；安全来自授权与业务校验，不是“使用 MCP”自动获得的。

17. 一个问题的完整示例

用户输入：

查询贵州茅台最近 5 个交易日的收盘价和成交量

1. 前端发送 `query + session_id + workspace`。
2. `FinQueryService` 生成 `user_id:session_id`、`task_id` 和各种上下文。
3. 预处理器判断为 `database_query`，提取“贵州茅台、交易日期、收盘价、成交量”。
4. BM25 与 Dense 召回字段，RRF 融合，Rerank 筛选。
5. 构建只含 `stock_daily` 必要字段的最小 Schema 图。
6. Agent 通过 `list_tools()` 得知 `query_trade_data`。
7. Agent 生成 SQL 并调用 MCP 工具。
8. DuckDB 校验权限和只读规则后执行。
9. `ResponseGenerator` 生成标题与分析。
10. `ResultBuilder.completed()` 返回前端。
11. `SessionContext.remember()` 把完成轮次加入当前 Session。

示意 SQL：

```
SELECT trade_date, close, vol
FROM stock_daily
WHERE ts_code = '600519.SH'
ORDER BY trade_date DESC
LIMIT 5;
```

18. 重点源码索引

学习主题	文件	重点符号
前端请求	App.vue	<code>submit()</code>
Session 创建/删除	App.vue	<code>createConversation()</code> 等
HTTP/Token	api.ts	<code>api.query()</code>
Workspace 类型	types.ts	<code>WorkspaceConfig</code>
API 模型	models.py	<code>QueryRequest</code> 等
服务入口	finquery_service.py	<code>submit()</code>
Payload	finquery_service.py	<code>_payload()</code>
Session 上下文	session_context.py	<code>tasks</code> 、 <code>session_tasks</code>
短期摘要	short_term_memory.py	<code>ShortTermMemory</code>
Session Archive	session_archive.py	<code>SessionArchive</code>
长期保存	memory_store.py	<code>MemoryStore</code>
State	state.py	<code>QueryState</code>
图定义	query_graph.py	<code>_compile()</code>
图运行	query_graph.py	<code>invoke()</code>
路由/改写	preprocessing.py	<code>prepare()</code>
Schema 召回	retrieval/service.py	<code>retrieve()</code>
BM25/Dense	retrieval/store.py	两种搜索函数
RRF/Rerank	retrieval/service.py	<code>fused</code> 、 <code>rerank()</code>
同义词	trade_data.py	<code>SYNONYMS</code>
Schema 图	retrieval/graph.py	<code>build()</code>
SQL Agent	single_database_agent.py	<code>prepare()</code>
已有数据分析	data_qa_agent.py	<code>run()</code>
MCP Client	client.py	<code>list_tools()</code> 、 <code>call_tool()</code>
MCP Server	server.py	<code>create_local_mcp_server()</code>
数据库工具	database_tools.py	<code>build_database_query_tool()</code>
SQL 安全/执行	duckdb_engine.py	<code>execute()</code> 、 <code>_validate_sql()</code>
结果解释	response_generator.py	<code>finalize()</code>

学习主题	文件	重点符号
Agent 契约	database_query/SKILL.md	工具与输出动作约束

19. 推荐阅读顺序

1. `App.vue::submit()`：前端发送什么。
2. `api.ts::query()`：HTTP 请求。
3. `models.py`：API 契约。
4. `FinQueryService.submit()` / `_payload()`：上下文组装。
5. `workflows/state.py`：State 字段。
6. `QueryWorkflow._compile()`：整张图。
7. `RequestPreprocessor.prepare()`：三种路由。
8. `SchemaIndex.retrieve()`：BM25、Dense、RRF、Rerank。
9. `SchemaGraphBuilder.build()`：最小 Schema 图。
10. `SingleDatabaseAgent.prepare()`：模型、MCP 和 SQL。
11. `mcp_runtime`：工具发现与调用。
12. `DuckDbEngine`：SQL 安全与执行。
13. `ResponseGenerator` / `ResultBuilder`：最终结果。
14. `SessionContext` / `ShortTermMemory` / `MemoryStore`：记忆层次。

20. 最容易混淆的点

- 召回不是最终回答，只是缩小候选范围。
- Dense 检索不等于使用向量数据库；当前是本地全量点积。
- MCP 不等于 Agent-to-Agent；Server 可以只是数据库/API 适配器。
- 工具描述用于可发现性，不等于授权。
- 前端对话列表不等于 Agent 短期记忆。
- `session_id` 是多轮业务会话，`task_id` 是一次任务和 LangGraph checkpoint thread。
- LangGraph 支持循环，因此不一定是 DAG。
- 节点是业务处理器；Hook/Callback 是生命周期观察机制。
- `_execute_single_database()` 不是当前真正发起 SQL 的位置；SQL 已由 Agent 通过 MCP 调用。
- 最小 Schema 图不是越少越好，而是最小但足够。

21. 一句话总结

前端携带 Session 与显式 Workspace 发起问题，后端生成分层上下文并交给 LangGraph；LangGraph 先路由，再通过 BM25、Dense、RRF 和 Rerank 召回字段，构造最小 Schema 图，让单库 Agent 通过进程内 MCP 安全调用 DuckDB，最后将结构化结果解释成回答，并把完成轮次写回当前 Session 的短期记忆。